



Machine Learning II

Data Science M.Sc.

Prof. Dr. Steffen Wagner
Berliner Hochschule für Technik

Winter term 2025/26



Classification

Support Vector Machines



Recap: Support Vector Classifier

- ▶ We have a classification problem with $K=2$ classes and p predictor variables.
- ▶ Each of the n observations can be treated as a p -dimensional vector $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{ip})$. We use $p=2$ for learning purposes.
- ▶ For convenience we label the two outcome classes using -1 and 1 , so $y_i \in \{-1, 1\}$.
- ▶ The search a boundary function $f(\mathbf{x})$ so that our classifier is determined by the sign of $f(\mathbf{x})$. I.e. for the test data point $\mathbf{x}^*$ the classifier is:

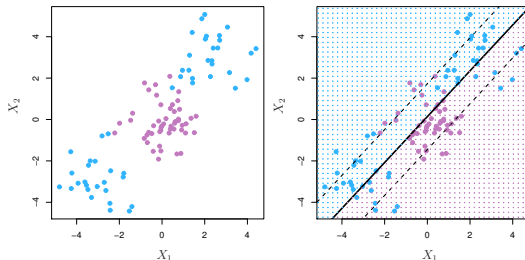
$$\hat{y}(\mathbf{x}^*) = \text{sign}(f(\mathbf{x}^*)) = \begin{cases} +1 & \text{if } f(\mathbf{x}^*) > 0 \\ -1 & \text{if } f(\mathbf{x}^*) < 0 \end{cases}$$

- ▶ Up to now we developed a method to separate the two classes in the training data using a linear boundary called a *hyperplane*.
- ▶ To either side of the boundary there is a margin. In the ideal case no data points lie in the margin and no data points lie on the wrong side of the boundary:
⇒ **Maximal margin classifier**.
- ▶ In practice it is better to use a *soft margin* so a few points are allowed in the margin or on the wrong side of the boundary, but at a cost, and the cost is held to a specified budget:
⇒ **Support vector classifier**



Where do linear SVMs fail?

There are situations when a linear boundary (= hyperplane) $f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ is insufficient.



We do need non-linear boundaries, i.e. non-linear SVMs.

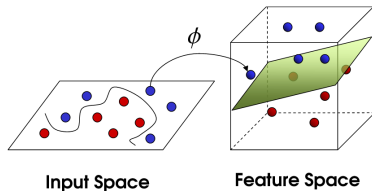


Feature space expansion

Hmmmhhh?! – How to apply a *linear method* to a *non-linear* problem?

Idea:

We apply the *linear method* to an **expanded feature space**¹ based on *non-linear* transformations of the original features (=input space).



source: <https://towardsdatascience.com/svm-feature-selection-and-kernels-840781cc1a6c>

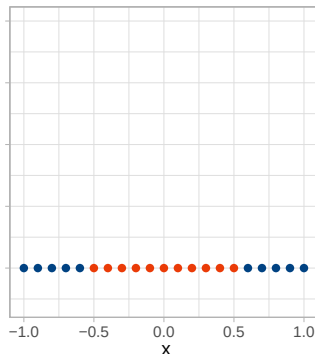
¹Actually, this is what we already did in spline smoothing by using a set of basis functions (= more features) instead only considering the original variable x .



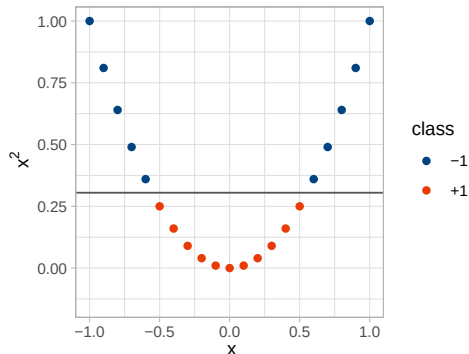
Feature space expansion (cont)

Example 1: from $\mathbb{R}$ to $\mathbb{R}^2$ via $x \rightarrow (x, x^2)$

no linear separation
we only consider x



linear separation
we consider x and x^2



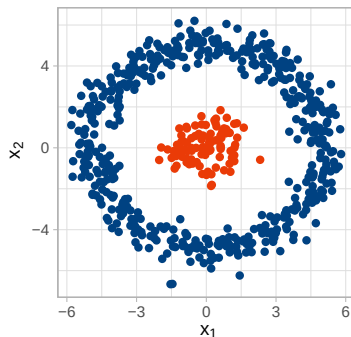


Feature space expansion (cont)

Example 2: from $\mathbb{R}^2$ to $\mathbb{R}^3$ via $(x_1, x_2) \rightarrow (x_1^2, \sqrt{2}x_1x_2, x_2^2)$

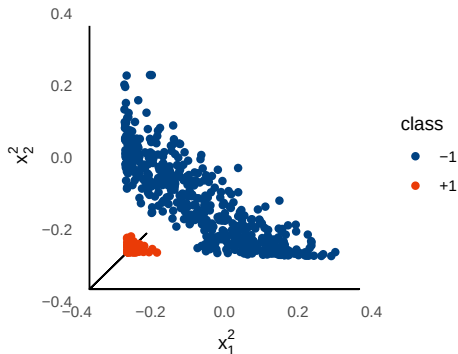
no linear separation

we consider x_1 and x_2



hyperplane separation

we consider x_1^2 , $\sqrt{2}x_1x_2$ and x_2^2





Support Vector Machines

The **support vector machine (SVM)** is an extension of the support vector support vector classifier that results from *enlarging the feature space* in a specific way, *using kernels*.

Keep in mind:

- ▶ Main idea: we want to enlarge the input feature space to accommodate to non-linear boundaries between classes.
- ▶ The kernel approach that we describe in the following is “only” an efficient computational approach for enacting this idea.



The kernel trick

Roughly speaking the kernel trick is useful if ...

1. ...*the data has more dimensions than samples.*
2. ...*the problem requires unknown(!) non-linear transformations of the data.*

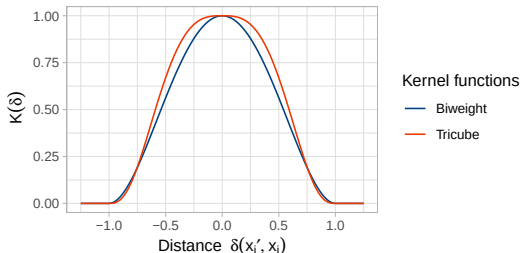
Both properties are highly appreciated as we want to extend the input feature space considering additional components based on non-linear transformations of existing features.



Recap: Kernel functions in *loess* smoothing

Kernel definition (for *loess* smoothing):

The kernel $K(\delta)$ is a function that quantifies the similarity of two observations $x_{i'}$ and x_i based on their distance.



- ▶ Here the similarity measure is distance.
- ▶ The fitted *loess* smoothing function $\hat{f}(x)$ is a weighted sum of kernels.



Kernel functions

General kernel definition:

A kernel $K(\mathbf{x}_{i'}, \mathbf{x}_i)$ is a function that **quantifies the similarity** of two observations $\mathbf{x}_{i'}$ and $\mathbf{x}_i$ **based on their inner (= scalar) product** $\langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle$:

$$K(\mathbf{x}_{i'}, \mathbf{x}_i) = K(\langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle)$$

- ▶ Considering two observations $\mathbf{x}_i$ and $\mathbf{x}_{i'}$ as vectors in p -dim feature space, we can use the cosine of the angle θ between their vectors as similarity measure, the so-called *cosine* similarity $\in [-1, 1]$.

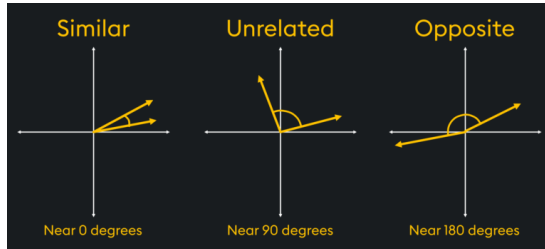
$$\text{cosine similarity} = \cos(\theta) = \frac{\mathbf{x}_{i'}^\top \cdot \mathbf{x}_i}{\|\mathbf{x}_{i'}\| \cdot \|\mathbf{x}_i\|} = \frac{\langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle}{\|\mathbf{x}_{i'}\| \cdot \|\mathbf{x}_i\|}$$

- ▶ Having unit vectors, the cosine similarity equals the inner product of the two vectors $\mathbf{x}_{i'}$ and $\mathbf{x}_i$

$$\cos(\theta) = \langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle = \sum_{j=1}^p x_{i'}' x_j$$



Cosine similarity



SOURCE: <https://kdb.ai/learning-hub/articles/methods-of-similarity/>



Kernel functions and SVM

To benefit from kernel functions (= applying the kernel trick) it is essential to express the *hyperplane* of the classifier in terms of kernels:

$$\begin{aligned} f(\mathbf{x}) &= \beta_0 + \sum_{j=1}^p \beta_j x_j \\ &= \beta_0 + \sum_{i=1}^n \alpha_i \underbrace{\langle \mathbf{x}, \mathbf{x}_i \rangle}_{K(\mathbf{x}, \mathbf{x}_i)} \quad \text{with } \mathbf{x} = (x_2, \dots, x_p)^\top. \end{aligned}$$

- ▶ By expanding each of the inner products, it is easy to see that $f(\mathbf{x})$ is a linear function of the coordinates of $\mathbf{x}$. Doing so also establishes the correspondence between the α_i and the original parameters β_i .
- ▶ To estimate the parameters $\beta_0, \alpha_1, \dots, \alpha_n$ we first obtain the inner products between all pairs of data points.
- ▶ As the inner product is symmetric we need to calculate $\frac{1}{2}n(n+1)$ inner products.
In addition, each inner product requires order p calculations.
- ▶ This is expensive, but these values can be calculated just once at the start of the algorithm and do not need to be recalculated with each iteration.



OMG: $n + 1$ parameters ?!

The alternative representation of the linear classifier function is *over-parametrised*:

- ▶ We now have $n+1$ parameters to estimate instead of $p+1$ parameters β_i .
- ▶ **but:** the fitted α_i coefficients will be zero for the non-support vectors².

This means, we can re-express the boundary classifier as

$$f(\mathbf{x}) = \beta_0 + \sum_{i \in S} \alpha_i \langle \mathbf{x}, \mathbf{x}_i \rangle ,$$

where S is the set of support vectors.

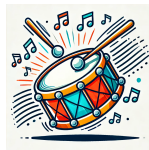
- ▶ As we expect that the number of support vectors is small compared to n the numerical complexity of the boundary function is reduced significantly.
- ▶ **but:** In the beginning we don't know the support vectors. This means $\frac{n(n-1)}{2}$ inner products have to be calculated each requiring order p calculations.

²Why? Think about it ...



The kernel trick

Kernel trick: Save time/space by computing the value of $K(\mathbf{x}_{i'}, \mathbf{x}_i)$ in the original feature space instead of calculating $\langle \phi(\mathbf{x}_{i'}), \phi(\mathbf{x}_i) \rangle$ in the expanded feature space³.



This works because $K(\mathbf{x}_{i'}, \mathbf{x}_i) = K(\underbrace{\langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle}_{\text{input space}}) = \underbrace{\langle \phi(\mathbf{x}_{i'}), \phi(\mathbf{x}_i) \rangle}_{\text{expanded feature space}} .$

Different coefficients do not really matter, only the identical linear structure regarding data point similarity in both sides of the equation is important.

³can even be infinite



*"Young man, in mathematics you don't understand things.
You just get used to them."*

John von Neumann (1903 - 1957)



photo: https://en.wikipedia.org/wiki/John_von_Neumann



Linear vs non-linear SVM

It's all about hyperplane specifications:

- ▶ Linear SVM using a linear kernel $K_{\text{lin}}(\mathbf{x}, \mathbf{x}_i)$:

$$\text{hyperplane:} \quad f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n \alpha_i K_{\text{lin}}(\mathbf{x}, \mathbf{x}_i)$$

- ▶ Non-linear SVM using a non-linear kernel $K_{\text{nl}}(\mathbf{x}, \mathbf{x}_i)$:

$$\text{hyperplane:} \quad f(\mathbf{x}) = \beta_0 + \sum_{i=1}^n \alpha_i K_{\text{nl}}(\mathbf{x}, \mathbf{x}_i)$$

Important:

For a non-linear boundary, we *just* replace the linear kernel by a non-linear kernel.



(Non-linear) SVM algorithm

- ▶ The extra computation at the start, when calculating the pairwise kernel values $K(\mathbf{x}_i, \mathbf{x}_j)$, is offset by allowing a very large extended feature space to be implicitly explored, while remaining computationally feasible.
- ▶ The same algorithm is used for linear and non-linear boundaries, the only difference is the type of kernel function to use.
- ▶ The same concept of the margin is used in linear and non-linear SVMs, i.e. the same support vector classifier algorithm in last week's lecture can be applied:

$$\max_{\beta_0, \alpha_i, \epsilon_i} M, \text{ such that } y_i \left(\beta_0 + \sum_{j=1}^n \alpha_j K(\mathbf{x}_i, \mathbf{x}_j) \right) \geq M(1 - \epsilon_i)$$

subject to the constraints $\sum_{i=1}^n \alpha_i^2 = 1$, $\epsilon_i \geq 0$ and $\sum_{i=1}^n \epsilon_i < C$,

where $C \geq 0$ is the cost parameter and $i=1, \dots, n$.



Non-linear kernel functions

For a curved boundary use a **polynomial kernel** function of degree $d > 1$

$$K(\mathbf{x}'_i, \mathbf{x}_i) = (c + \langle \mathbf{x}'_i, \mathbf{x}_i \rangle)^d .$$

- The constant c is not critical to the procedure, it is just chosen to keep each term positive. If the data vectors have length 1 then using $c=1$ is sufficient.



Non-linear kernel functions

Another common choice is the **radial kernel**

$$K(\mathbf{x}_{i'}, \mathbf{x}_i) = \exp(-\gamma \|\mathbf{x}_{i'} - \mathbf{x}_i\|^2) = \exp\left(-\gamma \sum_{i=1}^p (x'_i - x_i)^2\right) \quad \text{with } \gamma > 0.$$

Remarks:

- ▶ This kernel considers the squared Euclidean distance between points.
- ▶ Large distances lead to tiny values in $K()$ due to the exponential decay.
- ▶ Therefore the radial kernel has very local behaviour: only observations nearby have an effect on the learned class label.
- ▶ The Euclidean distance can be re-expressed using by $\langle \mathbf{x}_{i'}, \mathbf{x}_i \rangle$.
- ▶ The feature space for the radial kernel is *implicit* and *infinite-dimensional*.

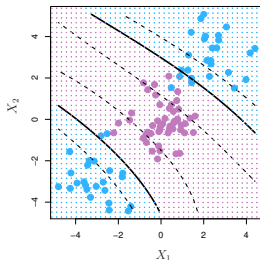
Details: <https://pages.cs.wisc.edu/~matthewb/pages/notes/pdf/svms/RBFKernel.pdf>



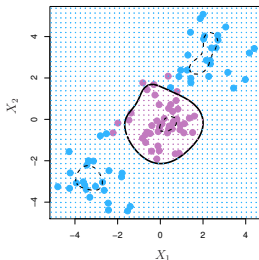
Where do non-linear SVMs succeed?

The data introduced on slide 4 now modelled using SVMs with non-linear kernels:

polynomial kernel of degree 3



radial kernel



- ▶ The plots above show the non-linear boundary in the *original* input feature space $\mathbb{R}^2$ after the linear boundary in the transformed space has been obtained.
- ▶ No explicit transformation from the extended space into the original space is calculated. To find the regions and the boundaries, the predicted values are obtained for all the tiny blue and magenta points on the (X_1, X_2) -grid.



Classifying more than two classes

The SVM approach works well when the number of classes is two ($K=2$):

$$\hat{f}(\mathbf{x}) = \text{sign} \left(\beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(\mathbf{x}, \mathbf{x}_i) \right)$$

What to do if $K > 2$?

- ▶ Unfortunately there is no mathematical method to extend the classification to more than two classes, because there are only two sides of a hyperplane.
- ▶ A work-around lies in artificially creating multiple $K=2$ problems and running the SVM algorithm for each of them.



One-versus-one Classification

Let the possible class outcomes be $a_1, \dots, a_K$.

1. We create a subset of the data containing only those observations, in which $y_i \in \{a_1, a_2\}$ and define the SVM outcomes

$$y'_i = \begin{cases} +1 & \text{for } y_i = a_1 \\ -1 & \text{for } y_i = a_2 \end{cases}.$$

2. We fit the SVM model for this data subset.

Doing this for all pairwise combinations, we obtain $\frac{1}{2}K(K-1)$ SVM models.

- ▶ To calculate the predicted values, we summarise the classification given by each SVM model and use the majority vote method to get the overall classification.
- ▶ There are now $\frac{1}{2}K(K-1)$ SVM algorithms to run, however calculating the kernel function values $K(\mathbf{x}_i, \mathbf{x}_j)$ only needs to be done once on all the data, before taking each subset.



One-versus-all Classification

Let the possible class outcomes again be $a_1, \dots, a_K$.

1. We set a_1 to be the chosen class and the corresponding outcomes to be

$$y'_i = \begin{cases} +1 & \text{for } y_i = a_1 \\ -1 & \text{otherwise} \end{cases}$$

2. We fit the SVM model for this re-labelled data set and obtain the SVM boundary function $\hat{f}_1(\mathbf{x})$.

We repeat so that each class $a_1, \dots, a_K$ is once the chosen class and obtain K boundary functions $\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \dots, \hat{f}_K(\mathbf{x})$.

- The predicted value at the point $\mathbf{x}'$ is the class giving the largest value of $\hat{f}_k(\mathbf{x}')$, i.e.

$$\hat{y}' = \arg \max \{ \hat{f}_k(\mathbf{x}') \}.$$

- We are choosing the binary classifier for which $\mathbf{x}'$ is “furthest away” from the boundary. We can think of this as being the class, that we have the most confidence in.



Let's take a step back ...

...and look at **SVMs in the context with other machine learning methods**⁴

- ▶ When SVMs came up in the mid-1990s they were quite surprising for the statistical community due to their good performance and the fact that the underlying approach seemed both novel and distinctly different from classical statistical approaches like logistic regression or LDA.
- ▶ The idea of using a kernel to expand the feature space in order to accommodate non-linear class boundaries appeared to be a unique and valuable characteristic.

These seemingly fundamental differences have been analysed in great detail, with the result that the various approaches can now be formulated and understood within a common framework, i.e. **the bigger picture**.

⁴see section 9.5 in *James et al.* (p. 386)



The bigger picture

The SVM as a Penalization Method:

The fitting criterion for a SVM can be formulated in its *dual*⁵ representation with $f(\mathbf{x}) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$ as

$$\min_{\beta} \left\{ \underbrace{\sum_{i=1}^n \max[0, 1 - y_i f(x_i)]}_{= \text{model mismatch}} + \lambda \underbrace{\sum_{j=1}^p \beta_j^2}_{\text{penalty}} \right\}$$

- ▶ The penalty term is the ridge penalty term you know from *penalized regression* discussed in ML1.
- ▶ You also know that λ controls the bias-variance-trade-off.

⁵[https://en.wikipedia.org/wiki/Duality_\(optimization\)](https://en.wikipedia.org/wiki/Duality_(optimization))



Let's take another step back ...

...and formulate a very *general model fitting criterion* as

$$\min_{\beta} \left\{ \underbrace{L(\mathbf{X}, \mathbf{y}, \beta)}_{= \text{model mismatch}} + \underbrace{\lambda P(\beta)}_{\text{penalty}} \right\}$$

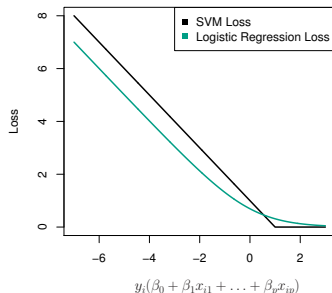
We introduce ...

- ▶ the **loss function** $L(\mathbf{X}, \mathbf{y}, \beta)$ that quantifies the extent to which the model (as a function of β) does not fit the data $(\mathbf{X}, \mathbf{y})$ and
- ▶ the **penalty** $P(\beta)$ that penalizes overfitting.



SVM vs *penalized* logistic regression

The “only” difference between SVMs and penalized logistic regression (LR) for classification is the difference in the model specific loss functions:



$$\text{SVM loss: } L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^n \max[0, 1 - y_i f(x_i)]$$

$$\text{LR loss: } L(\mathbf{X}, \mathbf{y}, \beta) = \sum_{i=1}^n \log[1 + e^{-y_i f(x_i)}]$$

Since the differences aren't that huge, usually classification results don't differ too much.



SVM vs *penalized* logistic regression

- ▶ James et al recommends: *When the classes are well separated, SVMs tend to behave better than logistic regression; in more overlapping regimes, logistic regression is often preferred.*
- ▶ An advantage with logistic regression is that the model formula can help us to understand the structure in the data. This is not easy to do from the results of a SVM model, which concentrates on prediction.
- ▶ The logistic regression approach can be generalised to non-linear classification using the non-linear regression techniques discussed in the non-linear regression lessons.
- ▶ But still: SVMs are *much* more widely used for non-linear classifiers than logistic regression.



What about SVM regression?

Yeah, what about it ???

This will be a beautiful discussion in class and you might be surprised that we only have to adapt the loss function to switch from SVM classification to SVM regression.⁶

⁶If you can't wait for class or missed it, please see *Hasties et al.* section 12.3.6 (p. 434) or *James et al.* end of section 9.5 (p. 388).